

**FACTORY
METHOD
PATTERN**

Factory Method Pattern

Factory Method Pattern (C#)

Vấn đề

Các vấn đề trong hệ thống thông báo



Phải sửa code

Khi thêm loại thông báo mới, lập trình viên phải sửa đổi mã nguồn, điều này gây khó khăn cho việc bảo trì và nâng cấp hệ thống.



Vi phạm nguyên tắc

Việc phải thay đổi mã nguồn khi thêm loại mới vi phạm nguyên tắc **Open/Closed**; nguyên tắc này yêu cầu hệ thống nên mở cho việc mở rộng nhưng đóng cho việc sửa đổi.



Phụ thuộc chặt

Hệ thống này phụ thuộc vào các lớp cụ thể, làm giảm tính linh hoạt và khả năng tái sử dụng mã nguồn trong tương lai.



Ý tưởng của Factory Method



Ý tưởng Factory Method

Khái niệm về phương thức tạo object

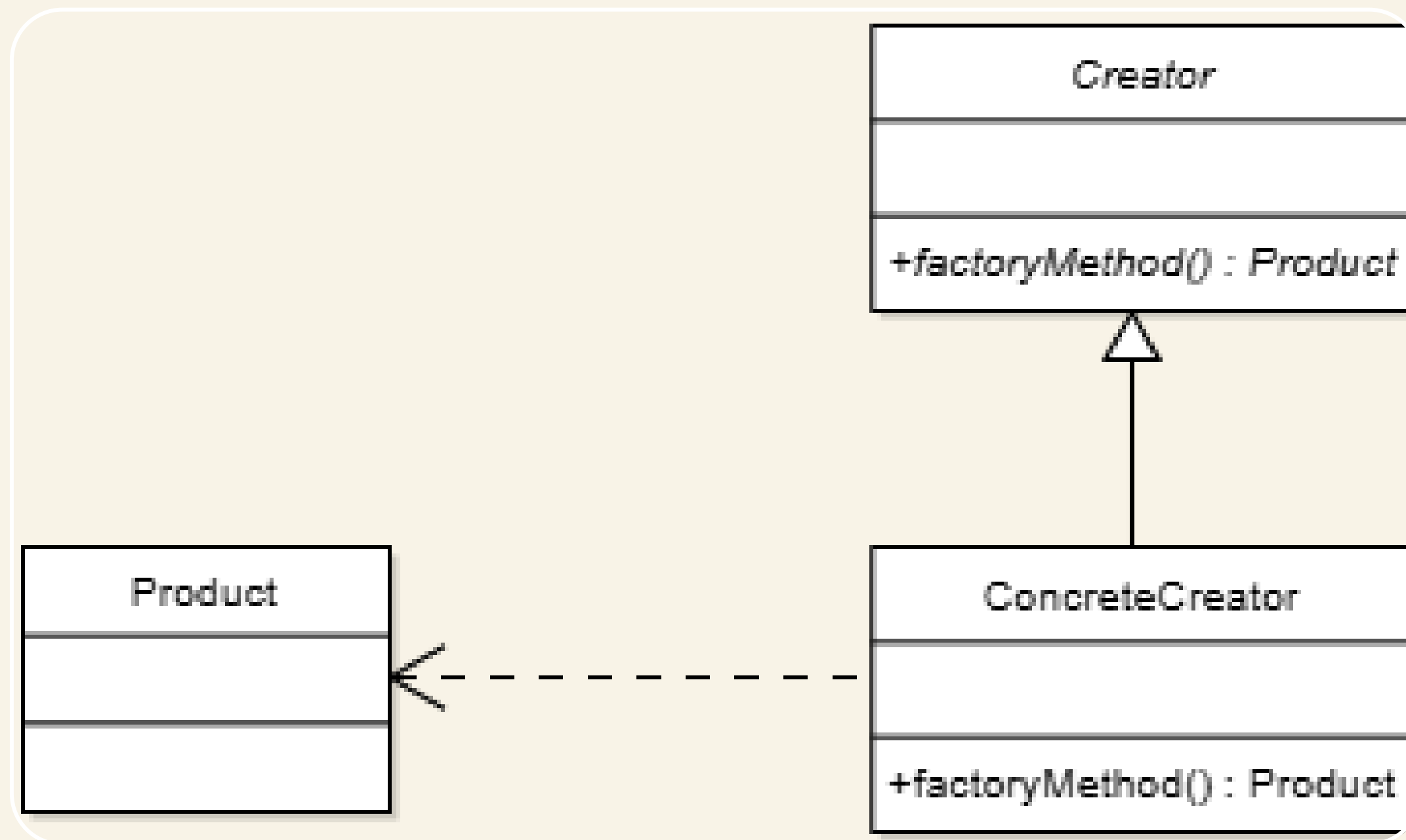
Định nghĩa phương thức

Phương thức tạo object được định nghĩa trong lớp cha, cho phép các lớp con tùy chỉnh việc tạo đối tượng cụ thể theo nhu cầu của mình.

Lớp con override

Các lớp con sẽ override phương thức này để cung cấp cài đặt cụ thể cho từng loại đối tượng, giúp tách biệt logics tạo object với việc sử dụng.

Cấu trúc của Pattern



Cấu trúc Pattern

Các thành phần chính trong Factory Method

Product

Product là interface chung cho tất cả các sản phẩm cụ thể, định nghĩa các phương thức mà các lớp cài đặt phải thực hiện để tương tác.

ConcreteProduct

ConcreteProduct là các lớp cụ thể cài đặt interface Product, cung cấp các hành vi cụ thể cho từng loại sản phẩm mà Factory Method tạo ra.

Creator

Creator là lớp cha chứa phương thức tạo Object, định nghĩa phương thức Factory Method mà các lớp con sẽ ghi đè để tạo sản phẩm cụ thể.

UML Cấu trúc

Diagram UML thể hiện cấu trúc của Factory Method, bao gồm các lớp và mối quan hệ giữa chúng.

Factory Method

Aftxstory Wedehurst



Product Interface

Product Interface

```
public interface INotification
{
    void Send();
}
```



Định nghĩa các phương thức cho tất cả các loại thông báo, đảm bảo hành vi nhất quán giữa các triển khai trong mô hình Factory Method.

Ví dụ C# — Sản phẩm cụ thể

Hai lớp cụ thể EmailNotification và SmsNotification triển khai INotification, mỗi lớp đều ghi đè phương thức Send().

```
public interface INotification
{
    void Send();
}

public class EmailNotification : INotification
{
    public override void Send()
    {
        Console.WriteLine("Sending Email Notification...");
    }
}

public class SmsNotification : INotification
{
    public override void Send()
    {
        Console.WriteLine("Sending SMS Notification...");
    }
}
```



Sending Email Notification...



Sending SMS Notification...

Ví dụ C# — Creator

```
public abstract class NotificationCreator
{
    public abstract INotification CreateNotification();

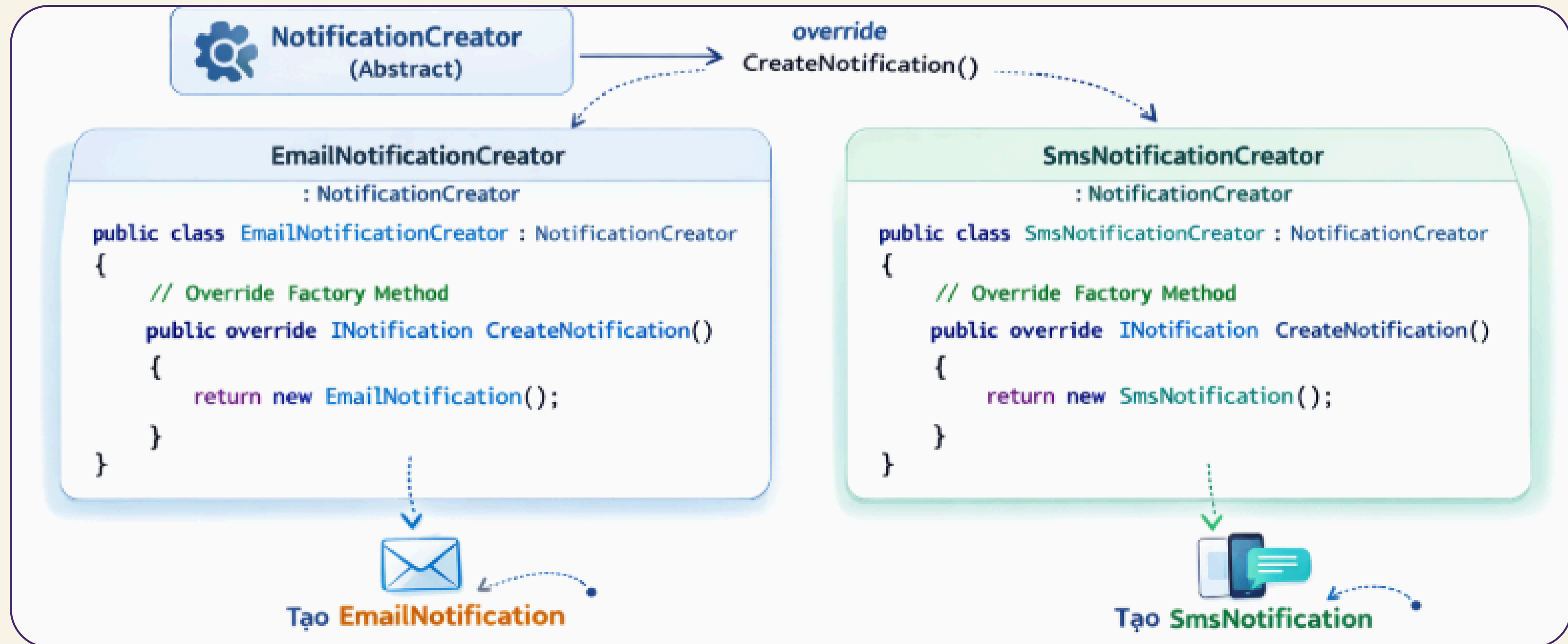
    public void SendNotification()
    {
        INotification notification = CreateNotification();
        notification.Send();
    }
}
```



Định nghĩa phương thức Factory, cho phép tạo đối tượng thông báo cụ thể.

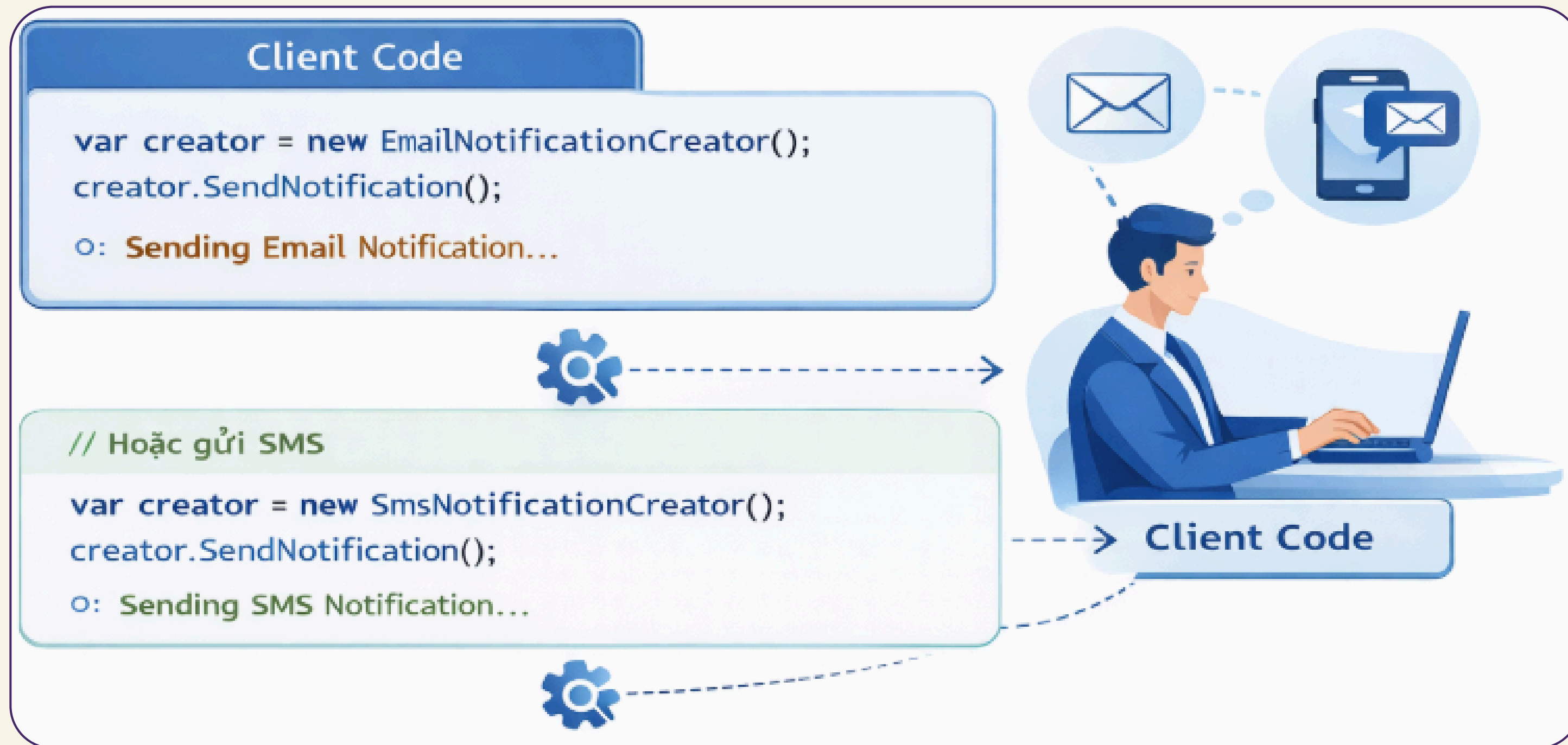
Concrete Creators

Các lớp ConcreteCreator sẽ override phương thức Factory Method để tạo ra đối tượng cụ thể như EmailNotification và SmsNotification.



Ví dụ C# — Client Code

Client sử dụng Creator để gửi thông báo mà không cần biết đối tượng cụ thể nào được tạo ra.



Ưu điểm

Những lợi ích của Factory Method

Tuân thủ nguyên tắc

Factory Method tuân thủ nguyên tắc Open/Closed, cho phép hệ thống mở rộng mà không cần sửa đổi mã nguồn hiện tại, từ đó nâng cao tính ổn định và an toàn.

Giảm coupling

Bằng cách tách biệt việc tạo đối tượng và sử dụng đối tượng, Factory Method giảm thiểu sự phụ thuộc giữa các lớp, làm cho mã dễ bảo trì hơn.

Dễ dàng mở rộng

Khi cần thêm loại đối tượng mới, chỉ cần tạo lớp con mới mà không ảnh hưởng đến các lớp đã tồn tại, giúp dễ dàng mở rộng hệ thống trong tương lai.

Đặt câu hỏi

THANK
YOU FOR
LISTENING

